

**Q 1.What are the key differences between Procedural Programming and ObjectOrientedProgramming (OOP)?**

**A:Procedural Programming :**

- Emphasis is on doing things.
- Large programs are broken into small ones, known as functions.
- Most of the functions share global data.
- Data move openly around the system from function to function.
- Follows Top-Down approach in program design.

**Object Oriented Programming :**

- Emphasis on data rather than Procedure.
- Programs are divided into objects.
- Data is hidden and can't be accessed by external functions.
- Objects may communicate with each other through functions.
- Follow Bottom-Up approach in program design.

**Q 2.List and explain the main advantages of OOP over POP.**

**A:Main Advantages of OOP over POP**

**1. Encapsulation (Data Hiding)**

- Combines data and functions into a single unit called a class
- Protects data from unauthorized access

**2. Reusability**

- Uses inheritance to reuse existing code
- Avoids rewriting the same code

**3. Modularity**

- Programs are divided into independent objects
- Each object handles a specific task

**4. Easy Maintenance**

- Changes in one object do not affect others
- Suitable for large applications

## 5. Better Real-World Modeling

- Represents real-world entities using objects and classes

## 6. Abstraction

- Shows only essential features
- Hides complex implementation details

## 7. Polymorphism

- Same function name behaves differently in different contexts

## 8. Improved Security

- Data is accessed through methods, not directly

**Q 3.Explain the steps involved in setting up a C++ development environment.**

### **A:1. Install a C++ Compiler**

A compiler converts C++ source code into an executable program.

Common Compilers

- GCC (GNU Compiler Collection) – Widely used on Linux and macOS
- MinGW / MinGW-w64 – GCC for Windows
- MSVC (Microsoft Visual C++) – Comes with Visual Studio (Windows)
- Clang – Popular on macOS and Linux

### **2. Set Up Environment Variables (if required)**

- On **Windows**, add the compiler's `bin` directory to the **PATH** variable so the system can recognize commands like `g++`.
- On **Linux/macOS**, this is usually done automatically.

### **3. Install an Integrated Development Environment (IDE) or Code Editor**

An IDE provides code editing, compilation, debugging, and project management.

## Popular IDEs

- Visual Studio (Windows)
- Code::Blocks
- Dev-C++
- CLion
- Xcode (macOS)

## Lightweight Editors (with compiler)

- VS Code (with C/C++ extension)
- Sublime Text
- Atom

## 4. Configure the IDE/Editor

- Select the installed compiler (GCC, MSVC, etc.)
- Set build and run configurations
- Install required extensions (e.g., C/C++ extension in VS Code)

## Q 4.What are the main input/output operations in C++? Provide examples.

### A:1. Output Operations

Used to display data on the screen.

cout (Standard Output Stream)

- Sends output to the console
- Uses the insertion operator <<

Example:

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    cout << "Hello, World!" << endl;
```

```
    cout << "Number: " << 10 << endl;
```

```
    return 0;
```

```
}
```

## 2. Input Operations

Used to read data from the keyboard.

cin (Standard Input Stream)

- Reads input from the user
- Uses the extraction operator >>

Example:

```
#include <iostream>

using namespace std;

int main() {

    int age;

    cout << "Enter your age: ";

    cin >> age;

    cout << "Age is: " << age << endl;

    return 0;

}
```

## Q 5.What are the different data types available in C++? Explain with examples.

**A:**In C++, data types specify the type of data a variable can store. They are broadly classified into basic (primitive), derived, user-defined, and type modifiers.

### 1. Basic (Primitive) Data Types

These are the fundamental data types.

a) int

Stores integer values.

```
int age = 20;
```

#### **b) float**

Stores decimal numbers with single precision.

```
float price = 45.75;
```

#### **c) double**

Stores decimal numbers with double precision.

```
double salary = 45890.50;
```

#### **d) char**

Stores a single character.

```
char grade = 'A';
```

#### **e) bool**

Stores boolean values (true or false).

```
bool isPassed = true;
```

#### **f) void**

Represents no value (used for functions that return nothing).

```
void showMessage() {  
    cout << "Hello!";  
}
```

## **2. Derived Data Types**

Derived from basic data types.

#### **a) Array**

Stores multiple values of the same type.

```
int numbers[5] = {1, 2, 3, 4, 5};
```

### **b) Pointer**

Stores the address of a variable.

```
int x = 10;
```

```
int *ptr = &x;
```

### **c) Reference**

An alias for another variable.

```
int a = 5;
```

```
int &ref = a;
```

### **d) Function**

A block of code that performs a specific task.

```
int add(int a, int b) {  
    return a + b;  
}
```

## **3. User-Defined Data Types**

Created by the programmer.

### **a) struct**

Groups variables of different types.

```
struct Student {  
    int roll;  
    string name;  
};
```

### **b) union**

Stores different data types in the same memory location.

```
union Data {  
    int i;  
    float f;  
};
```

### **c) enum**

Defines a set of named integer constants.

```
enum Day {Monday, Tuesday, Wednesday};
```

### **d) class**

Blueprint for creating objects.

```
class Car {  
public:  
    string brand;  
    int speed;  
};
```

**Q 6.Explain the difference between implicit and explicit type conversion in C++.**

**A:**

| Feature      | Implicit Conversion | Explicit Conversion |
|--------------|---------------------|---------------------|
| Performed by | Compiler            | Programmer          |

|           |                  |               |
|-----------|------------------|---------------|
| Control   | Automatic        | Manual        |
| Data loss | Usually no       | Possible      |
| Syntax    | No cast required | Cast required |
| Safety    | Safer            | Less safe     |

**Q 7.What are the different types of operators in C++? Provide examples of each.**

**A:1. Arithmetic Operators**

Used to perform mathematical calculations.

| Operator | Operation      |
|----------|----------------|
| +        | Addition       |
| -        | Subtraction    |
| *        | Multiplication |
| /        | Division       |



%            Modulus

**Example:**

```
int a = 10, b = 3;

cout << "Addition: " << a + b ;

cout << "Subtraction: " << a - b;

cout << "Multiplication: " << a * b;

cout << "Division: " << a / b;

cout << "Modulus: " << a % b;
```

**2. Relational (Comparison) Operators**

Used to compare values.

| Operator | Meaning |
|----------|---------|
|----------|---------|

|    |          |
|----|----------|
| == | Equal to |
|----|----------|

|    |              |
|----|--------------|
| != | Not equal to |
|----|--------------|

|   |              |
|---|--------------|
| > | Greater than |
|---|--------------|

|   |           |
|---|-----------|
| < | Less than |
|---|-----------|

|    |                          |
|----|--------------------------|
| >= | Greater than or<br>equal |
|----|--------------------------|

|    |                       |
|----|-----------------------|
| <= | Less than or<br>equal |
|----|-----------------------|

**Example:**

```
int x = 5, y = 10;
```

```
cout << (x < y);
```

### 3. Logical Operators

Used to combine conditions.

| Operator | Meaning |
|----------|---------|
|----------|---------|

|    |                |
|----|----------------|
| && | Logical<br>AND |
|----|----------------|

|   |               |
|---|---------------|
| ` | Logical<br>OR |
|---|---------------|

|   |                |
|---|----------------|
| ! | Logical<br>NOT |
|---|----------------|

**Example:**

```
if (x > 0 && y > 0) {
```

```
    cout << "Both positive";
```

```
}
```

### 4. Assignment Operators

Used to assign values.

| Operator | Example |
|----------|---------|
|----------|---------|

|   |       |
|---|-------|
| = | a = 5 |
|---|-------|

|    |       |
|----|-------|
| += | a +=2 |
|----|-------|

|    |       |
|----|-------|
| -= | a -=2 |
|----|-------|

|    |       |
|----|-------|
| *= | a *=2 |
|----|-------|

|    |       |
|----|-------|
| /= | a /=2 |
|----|-------|

**Example:**

```
int a = 10;
```

```
a += 5;
```

## 5. Bitwise Operators

Operate on bits.

| Operator | Meaning |
|----------|---------|
|----------|---------|

|   |     |
|---|-----|
| & | AND |
|---|-----|

, ,

^ XOR

~ NOT

<< Left shift

>> Right  
shift

**Example:**

```
int a = 5, b = 3;
```

```
cout << (a & b);
```

**Q 8.Explain the purpose and use of constants and literals in C++.**

**A:1. Constants in C++**

**Purpose of Constants :**

- Prevent accidental modification of values
- Improve code readability
- Make programs easier to maintain
- Represent fixed values like PI, maximum limits, PIN numbers, etc.

**Using const Keyword :**

```
const float PI = 3.14;
```

**2. Literals in C++**

**Purpose of Literals :**

- Directly represent fixed values in source code
- Used to initialize variables or pass values to functions

## **Types of Literals with Examples**

### **a) Integer Literals :**

Whole numbers without decimal points.

```
int a = 10;
```

```
int b = 025;
```

```
int c = 0x1A;
```

### **b) Floating-Point Literals :**

Numbers with decimal points.

```
float x = 10.5f;
```

```
double y = 3.14159;
```

### **c) Character Literals :**

Single character enclosed in single quotes.

```
char grade = 'A';
```

### **d) String Literals :**

Sequence of characters enclosed in double quotes.

```
string name = "Sonali";
```

### **e) Boolean Literals**

Logical values.

```
bool isValid = true;
```

**Q 9.What are conditional statements in C++? Explain the if-else and switch statements.**

**A:**Conditional statements control the flow of execution depending on conditions.

Common conditional statements in C++:

- if
- if-else
- else-if ladder
- Switch

**If-else statements :**

Provides an alternative block if the condition is false.

**Syntax:**

```
if (condition) {  
    // executes if true  
}  
else {  
    // executes if false  
}
```

**Example:**

```
int num = 5;  
  
if (num % 2 == 0) {  
    cout << "Even number";  
}  
else {  
    cout << "Odd number";  
}
```

## **Switch statements :**

Used when a variable is compared against multiple constant values.

### **Syntax:**

```
switch (expression) {  
    case value1:  
        // code  
        break;  
    case value2:  
        // code  
        break;  
    default:  
        // code  
}
```

### **Example:**

```
int choice = 2;  
switch (choice) {  
    case 1:  
        cout << "Deposit";  
        break;  
    case 2:  
        cout << "Withdraw";  
        break;  
    case 3:
```

```
        cout << "Balance Enquiry";

        break;

    default:

        cout << "Invalid choice";

    }
```

**Q 10.What is the difference between for, while, and do-while loops in C++?**

**A:1. for Loop :**

Used when the number of iterations is known in advance.

**Syntax:**

```
for (initialization; condition; increment/decrement) {

    // loop body

}
```

**Example:**

```
for (int i = 1; i <= 5; i++) {

    cout << i << " ";

}
```

**Characteristics:**

- Initialization, condition, and update in one line
- Condition is checked before loop execution
- Best for count-controlled loops

**2. while Loop :**

Used when the number of iterations is not known beforehand.

**Syntax:**



```
while (condition) {  
    // loop body  
}
```

### **Example:**

```
int i = 1;  
  
while (i <= 5) {  
    cout << i << " ";  
    i++;  
}
```

### **Characteristics:**

- Condition checked before execution
- Loop may execute zero times
- Useful when input-dependent looping is required

### **3. do-while Loop :**

Used when the loop must execute at least once, regardless of the condition.

### **Syntax:**

```
do {  
    // loop body  
} while (condition);
```

### **Example:**

```
int i = 1;  
  
do {  
    cout << i << " ";
```

```
i++;  
} while (i <= 5);
```

### **Characteristics:**

- Condition checked after execution
- Executes at least once
- Useful for menu-driven programs

**Q 11. How are break and continue statements used in loops? Provide examples.**

### **A:1. break Statement :**

#### **Purpose**

- Immediately terminates the loop
- Control moves to the statement after the loop

#### **Used When**

- You want to stop the loop completely
- A required condition is met (e.g., correct PIN entered)

#### **Example**

```
#include <iostream>  
  
using namespace std;  
  
int main() {  
  
    for (int i = 1; i <= 5; i++) {  
  
        if (i == 3) {  
  
            break;  
  
        }  
  
        cout << i << " ";  
  
    }  
  
}
```

```
}  
    return 0;  
}
```

**Output:**

**1 2**

## **2. continue Statement :**

### **Purpose**

- Skips the current iteration
- Loop continues with the next iteration

### **Used When**

- You want to skip specific values
- Certain condition should not be processed

### **Example**

```
#include <iostream>  
  
using namespace std;  
  
int main() {  
    for (int i = 1; i <= 5; i++) {  
        if (i == 3) {  
            continue; // Skip when i is 3  
        }  
        cout << i << " ";  
    }  
    return 0;
```

```
}
```

**Output:**

**1 2 4 5**

**Q 12.Explain nested control structures with an example.**

**A:**Nested control structures mean using one control structure inside another.

**Example :**

```
#include <iostream>

using namespace std;

int main() {

    int age;

    char gender;

    cout << "Enter age: ";

    cin >> age;

    if (age >= 18) {

        cout << "Enter gender (M/F): ";

        cin >> gender;

        if (gender == 'F') {

            cout << "Eligible for women scholarship";

        }

    }

    else {

        cout << "Not eligible for women scholarship";

    }

}
```

```
else {  
    cout << "Not eligible due to age";  
}  
  
cout << "Not eligible due to age";  
  
}
```

**Q 13.What is a function in C++? Explain the concept of function declaration, definition, and calling.**

**A:**A function in C++ is a block of code that performs a specific task.

### **Concept of Function in C++**

A function has **three main parts**:

1. Function Declaration
2. Function Definition
3. Function Calling

#### **1. Function Declaration :**

The function declaration tells the compiler:

- function name
- return type
- number and type of parameters

It is usually written before `main()`.

#### **Syntax:**

```
return_type function_name(parameter_type parameter_name);
```

## 2. Function Definition

The function definition contains the actual code that performs the task.

### Syntax:

```
return_type function_name(parameter_type parameter_name) {  
    // function body  
}
```

## 3. Function Calling

The function call is used to execute the function.

When a function is called, control is transferred to the function.

### Example:

```
#include <iostream>  
using namespace std;  
  
// Function declaration  
int add(int, int);  
  
int main() {  
    int sum;  
  
    // Function calling  
    sum = add(5, 7);  
  
    cout << "Sum = " << sum;  
    return 0;  
}  
  
// Function definition  
int add(int a, int b) {  
    return a + b;  
}
```

### Output :

Sum = 12

**Q 14.What is the scope of variables in C++? Differentiate between local and global scope.**

**A:**The scope of a variable in C++ refers to the area of the program where the variable can be accessed or used.

Outside its scope, the variable cannot be used.

**Types of Variable Scope in C++**

1. Local Scope
2. Global Scope

**1. Local Scope**

A local variable is declared inside a function or a block ({ }).

**Characteristics:**

- Accessible only within that function or block
- Created when the function/block starts
- Destroyed when the function/block ends
- Has no effect outside its scope

**Example (Local Variable):**

```
#include <iostream>
using namespace std;

void display() {
    int x = 10; // local variable
    cout << x;
}

int main() {
    display();
    // cout << x;
    return 0;
}
```

## 2. Global Scope

A global variable is declared outside all functions, usually at the top of the program.

### Characteristics:

- Accessible throughout the program
- Lifetime is the entire program execution
- Uses memory as long as the program runs
- Can be accessed by all functions

### Example (Global Variable):

```
#include <iostream>
using namespace std;

int x = 50; // global variable

void show() {
    cout << x;
}

int main() {
    cout << x;
    show();
    return 0;
}
```

### Q 15.Explain recursion in C++ with an example.

**A:**Recursion is a programming technique in which a function calls itself to solve a problem.

#### Example:

```
#include <iostream>
using namespace std;
int factorial(int n) {
    if (n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}

int main() {
    int num;
```



```
    cout << "Enter a number: ";  
    cin >> num;  
    cout << "Factorial of " << num << " = " << factorial(num);  
    return 0;  
}
```

## **Q 16.What are function prototypes in C++? Why are they used?**

### **A:Function Prototypes in C++**

A function prototype in C++ is a declaration of a function that tells the compiler in advance about:

- the function name
- return type
- number of parameters
- data types of parameters

It is also called a function declaration.

### **Syntax of Function Prototype :**

**return\_type function\_name(parameter\_type1, parameter\_type2, ...);**

### **Example:**

```
int add(int, int);
```

## **Why Are Function Prototypes Used?**

Function prototypes are used to inform the compiler about a function before it is called.

### **Main Reasons:**

#### **1. Enables Function Calls Before Definition**

Without a prototype, a function must be defined before `main()`.

#### **2. Type Checking**

The compiler checks:

- correct number of arguments
  - correct data types
- This helps catch errors at compile time.

### **3. Improves Program Readability**

Prototypes act as a summary of functions at the top of the program.

### **4. Supports Large Programs**

In large projects, functions are often written in separate files.

Prototypes allow different files to communicate.

### **Q 17.What are arrays in C++? Explain the difference between single-dimensional and multi-dimensional arrays.**

#### **A:Arrays in C++**

An array in C++ is a collection of elements of the same data type stored in contiguous memory locations.

Arrays are used to store multiple values in a single variable, instead of declaring many separate variables.

#### **Syntax of an Array**

```
data_type array_name[size];
```

#### **Example:**

```
int marks[5];
```

#### **Single-Dimensional Array (1D Array)**

A single-dimensional array stores elements in a linear list.

#### **Example:**

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int marks[5] = {80, 75, 90, 85, 70};
```

```

    for (int i = 0; i < 5; i++) {

        cout << marks[i] << " ";

    }

    return 0;

}

```

### **Output:**

80 75 90 85 70

### **Characteristics:**

- Uses one index
- Simple to declare and access
- Suitable for storing lists (marks, salaries, prices)

## **Multi-Dimensional Array**

A multi-dimensional array stores data in the form of rows and columns (table or matrix form).

The most common is the two-dimensional array (2D).

### **Declaration:**

```
data_type array_name[rows][columns];
```

### **Example:**

```

#include <iostream>

using namespace std;

int main() {

    int matrix[2][3] = {

        {1, 2, 3},

        {4, 5, 6}
    }
}

```

```

};

for (int i = 0; i < 2; i++) {

    for (int j = 0; j < 3; j++) {

        cout << matrix[i][j] << " ";

    }

    cout << endl;

}

return 0;

}

```

### **Output:**

1 2 3

4 5 6

### **Characteristics:**

- Uses two or more indices
- Represents tables, matrices
- Requires nested loops for processing

## **Q 18.Explain string handling in C++ with examples.**

**A:**In C++, strings are used to store and manipulate text data.

C++ supports string handling in two ways:

1. C-style strings (character arrays)
2. C++ string class (from Standard Library)

### **1. C-Style Strings (Character Arrays)**

A C-style string is an array of characters ending with a null character '\0'.

**Declaration:**

```
char name[20];
```

**Example:**

```
#include <iostream>

#include <cstring>

using namespace std;

int main() {

    char str1[20] = "Hello";

    char str2[20] = "World";

    cout << "Length: " << strlen(str1);

    strcat(str1, str2); // Concatenation

    cout << "After concatenation: " << str1;

    strcpy(str2, str1); // Copy

    cout << "After copy: " << str2 << endl;

    return 0;

}
```

**2. C++ String Class (std::string)**

The string class is safer and easier than C-style strings.

**Header:**

```
#include <string>
```

**Example:**

```
#include <iostream>
```

```

#include <string>

using namespace std;

int main() {

    string s1 = "Hello";

    string s2 = "World";

    cout << "Length: " << s1.length();

    string s3 = s1 + " " + s2; // Concatenation

    cout << "Combined: " << s3;

    cout << "Substring: " << s3.substr(0, 5);

    return 0;

}

```

**Q 19.**How are arrays initialized in C++? Provide examples of both 1D and 2D arrays.

**A:1. Initialization of One-Dimensional (1D) Array :**

```
int a[n];
```

**Example:**

```

#include <iostream>

using namespace std;

int main() {

    int a[3];

    for (int i = 0; i < 3; i++) {

        cin >> a[i];

    }
}

```

```
    return 0;
}
```

## 2.Initialization of Two-Dimensional (2D) Array :

```
int [i] [j];
```

### Example :

```
#include <iostream>

using namespace std;

int main() {

    int mat[2][2];

    for (int i = 0; i < 2; i++) {

        for (int j = 0; j < 2; j++) {

            cin >> mat[i][j];

        }

    }

    return 0;

}
```

## Q 20.Explain string operations and functions in C++.

**A:**In C++, string operations are used to create, modify, compare, and manipulate text. These operations are mainly performed using:

1. C-style strings (char arrays)
2. C++ string class (std::string)

### 1. String Operations Using C-Style Strings

C-style strings are arrays of characters ending with '\0'.

String operations are performed using functions from the <cstring> header.

## Common C-String Functions

| Function | Purpose               | Example           |
|----------|-----------------------|-------------------|
| strlen() | Finds length          | strlen(str)       |
| strcpy() | Copies string         | strcpy(dest, src) |
| strcat() | Concatenates strings  | strcat(s1, s2)    |
| strcmp() | Compares strings      | strcmp(s1, s2)    |
| strlwr() | Converts to lowercase | strlwr(str)       |
| strupr() | Converts to uppercase | strupr(str)       |



### Example :

```
#include <iostream>

#include <cstring>

using namespace std;

int main() {

    char s1[20] = "Hello";

    char s2[20] = "World";

    cout << "Length of s1: " << strlen(s1);

    strcat(s1, s2);

    cout << "After concatenation: " << s1;

    cout << "Comparison: " << strcmp(s1, s2);

    return 0;

}
```

## 2. String Operations Using string Class

The string class provides **built-in member functions** that are safer and easier to use.

### Common string Functions

| Function          | Description             |
|-------------------|-------------------------|
| length() / size() | Returns length          |
| append()          | Adds string             |
| +                 | Concatenation operator  |
| compare()         | Compares strings        |
| substr()          | Extracts substring      |
| find()            | Finds position          |
| erase()           | Removes characters      |
| replace()         | Replaces part of string |

### Example :

```
#include <iostream>

#include <string>

using namespace std;
```

```

int main() {

    string s1 = "Hello";

    string s2 = "World";

    cout << "Length: " << s1.length();

    string s3 = s1 + " " + s2;

    cout << "Combined: " << s3;

    cout << "Substring: " << s3.substr(0, 5);

    return 0;

}

```

## **Q 21.Explain the key concepts of Object-Oriented Programming (OOP).**

**A:**Object-Oriented Programming (OOP) is a programming paradigm based on the concept of objects, which combine data (attributes) and functions (methods) that operate on the data.

OOP helps in writing programs that are modular, reusable, secure, and easy to maintain.

### **1. Class**

A class is a blueprint or template for creating objects.  
It defines data members and member functions.

```

class Student {

    int roll;

    char name[20];

public:

    void getData() { }

```

```
void display() { }  
};
```

## 2. Object

An object is an instance of a class.  
It represents a real-world entity.

Student s1; // s1 is an object

## 3. Encapsulation

Encapsulation is the process of wrapping data and functions together into a single unit (class).

→ Achieved using access specifiers: private, protected, public

```
class Account {  
private:  
    int balance;  
public:  
    void setBalance(int b) {  
        balance = b;  
    }  
};
```

## 4. Abstraction

Abstraction means hiding implementation details and showing only essential features.

→ Achieved using classes and access specifiers

```
class Shape {  
public:
```

```
void draw(); // only what to do, not how  
};
```

## 5. Inheritance

Inheritance allows one class to acquire the properties of another class.

```
class Animal {  
public:  
    void eat() {  
        cout << "Eating";  
    }  
};  
  
class Dog : public Animal {  
};
```

## 6. Polymorphism

Polymorphism means one function name, many forms.

### (a) Compile-Time Polymorphism

Achieved using function overloading or operator overloading.

```
int add(int a, int b);  
  
int add(int a, int b, int c);
```

### (b) Run-Time Polymorphism

Achieved using function overriding and virtual functions.

```
class Base {  
public:
```

```

virtual void show() {
    cout << "Base class";
}
};

class Derived : public Base {
public:
    void show() {
        cout << "Derived class";
    }
};

```

## 7. Data Hiding

Restricting access to data using private and protected.

→ Protects data from unauthorized access

**Q 22.What are classes and objects in C++? Provide an example.**

**A:Class :**

A class is a user-defined data type that acts as a blueprint for creating objects.  
It defines:

- Data members (variables)
- Member functions (functions)

**Syntax:**

```

class ClassName {
    // data members

```

```
        // member functions  
  
};
```

## **Object**

An object is an instance of a class.

It represents a real-world entity and occupies memory.

## **Syntax:**

```
ClassName objectName;
```

## **Example :**

```
#include <iostream>  
  
using namespace std;  
  
class Student {  
private:  
    int roll;  
    string name;  
public:  
    void input() {  
        cout << "Enter roll number: ";  
        cin >> roll;  
        cout << "Enter name: ";  
        cin >> name;  
    }  
    void display() {  
        cout << "Roll: " << roll;
```

```

        cout << "Name: " << name;

    }

};

int main() {

    Student s1; // Object creation

    s1.input(); // Calling member function

    s1.display();

    return 0;

}

```

**Q 23.What is inheritance in C++? Explain with an example.**

**A:**Inheritance is a mechanism of reusing and extending existing classes without modifying them, thus producing hierarchical relationships between them.

## **Syntax :**

```

class DerivedClass : access_specifier BaseClass {

    // members of derived class

};

```

## **Types of Inheritance in C++**

1. **Single Inheritance**  
One base class → one derived class
2. **Multiple Inheritance**  
One derived class inherits from multiple base classes
3. **Multilevel Inheritance**  
Inheritance in levels (A → B → C)
4. **Hierarchical Inheritance**  
One base class → multiple derived classes



## 5. Hybrid Inheritance

Combination of two or more types

### Example :

```
#include <iostream>

using namespace std;

// Base class

class Animal {

public:

    void eat() {

        cout << "Animal is eating";

    }

};

// Derived class

class Dog : public Animal {

public:

    void bark() {

        cout << "Dog is barking";

    }

};

int main() {

    Dog d;

    d.eat(); // inherited function

    d.bark(); // own function

    return 0;
```

```
}
```

**Output :**

Animal is eating

Dog is barking

**Q 24.What is encapsulation in C++? How is it achieved in classes?**

**A:**It means wrapping data (variables) and the functions that operate on that data into a single unit, called a class.

**How Is Encapsulation Achieved in C++?**

Encapsulation is achieved using:

1. Classes
2. Access specifiers (private, protected, public)

**Example :**

```
#include <iostream>

using namespace std;

class BankAccount {

private:

    int balance; // hidden data

public:

    void setBalance(int b) {

        balance = b; // controlled access

    }

    int getBalance() {

        return balance;

    }

}
```

```
};
```

```
int main() {
```

```
    BankAccount acc;
```

```
    acc.setBalance(5000);    // access via function
```

```
    cout << acc.getBalance();    // safe access
```

```
    return 0;
```

```
}
```



